Using and Creating Agents

Building Custom AI Assistants with GitHub Copilot

What We'll Cover

- **Agents vs Agentic AI** – clarifying terminology
- **Why use custom agents** – productivity benefits
- **Types of instruction files** – the Copilot ecosystem
- **Writing effective** `.agent.md` **files** – structure & best practices
- **Live example** – anatomy of a good agent file

Agentic AI vs Custom Agents

Term	Meaning
Agentic AI	AI systems that act autonomously—perceiving their environment, making decisions, and taking actions to achieve goals without constant human guidance
AI Agent	A specific implementation of agentic AI designed for a task—it observes, reasons, plans, and executes using available tools
Custom Agent	A configuration file (e.g., `.agent.md`) that shapes an AI agent's behavior, tools, and persona for specific workflows

Agentic AI is the *capability*; agents are *configured instances* of that capability.

Why Use Custom Agents?

- **Task-specific tools** – limit which tools are available (read-only for planning, full access for implementation)
- **Consistent behavior** – same instructions every time, no need to repeat yourself
- **Workflow handoffs** – chain agents together (Plan → Implement → Review)
- **Team sharing** – commit to repo so everyone uses the same configurations
- **Faster context** – AI understands your project conventions immediately

The Copilot Instruction Ecosystem

```
.github/
├── copilot-instructions.md      # Repository-wide instructions
├── instructions/
│   └── *.instructions.md        # Path-specific instructions
└── agents/
    └── *.agent.md               # Custom agents (personas)
```

File Type	Purpose
`copilot-instructions.md`	Applied to ALL Copilot interactions in the repo
`*.instructions.md`	Applied to specific file patterns (e.g., `**/*.ts`)
`*.agent.md`	Switchable personas with tools, models, and instructions

Custom Agent File Structure

```yaml
---
name: Planner
description: Generate implementation plans
tools: ['search', 'fetch', 'githubRepo']
model: Claude Sonnet 4
handoffs:
  - label: Start Implementation
    agent: implementation
    prompt: Implement the plan above.
---

# Planning Instructions

You are in planning mode. Generate a detailed
implementation plan. Do NOT make code edits.
```

Header Fields Explained

Field	Description
`name`	Display name (defaults to filename)
`description`	Shown as placeholder text in chat
`tools`	List of available tools for this agent
`model`	AI model to use (e.g., `Claude Sonnet 4`)
`handoffs`	Suggested next agents to switch to
`argument-hint`	Hint text guiding user input

Available Tools

Common built-in tools you can include:

- `search` – search files in workspace
- `fetch` – fetch web content
- `githubRepo` – search GitHub repositories
- `usages` – find code usages/references
- `terminalLastCommand` – get last terminal output
- `<mcp-server>/*` – all tools from an MCP server

Tip: Restrict tools for safety. A "Planner" agent shouldn't edit files!

Writing the Body (Instructions)

The body contains Markdown instructions that guide the AI's behavior:

```
# Code Review Instructions

You are a security-focused code reviewer.

## Your responsibilities:
- Check for SQL injection vulnerabilities
- Verify input validation
- Flag hardcoded secrets
- Suggest improvements, don't implement them

## Output format:
Provide findings as a numbered list with severity. Put this in a review.md file
```

Refer back to prompt engineering techniques for this!

Best Practices for Agent Instructions

1. **Be specific** – "Check for SQL injection" > "Review security"
2. **Define boundaries** – What should the agent NOT do?
3. **Set output format** – Markdown lists, tables, specific sections
4. **Include examples** – Show what good output looks like
5. **Keep it concise** – Instructions are prepended to every prompt
6. **Reference other files** – Use Markdown links to reuse instructions

Handoffs: Chaining Agents

Create guided workflows between agents:

```yaml
handoffs:
  - label: Start Implementation
    agent: implementation
    prompt: Implement the plan outlined above.
    send: false  # User must click to proceed
```

Example workflow:

1. **Planner** → generates implementation plan
2. **Implementation** → writes the code
3. **Reviewer** → checks for issues

Example: Solution Architect Agent

```
---
name: Solution Architect
description: Design technical solutions and architecture
tools: ['search', 'fetch', 'githubRepo', 'usages']
model: Claude Sonnet 4
handoffs:
  - label: Create Implementation Plan
    agent: planner
---

# Solution Architect Instructions

You are a senior solution architect. Your role is to:
- Analyze requirements and propose technical designs
- Consider scalability, security, and maintainability
- Reference existing patterns in the codebase
- Document trade-offs between approaches

Do NOT implement code. Focus on design decisions.
```

Creating Your First Agent

1. **In VS Code:** `Cmd+Shift+P` → "Chat: New Custom Agent"
2. **Choose location:**
   - **Workspace** (`.github/agents/`) – shared with team
   - **User profile** – personal, works everywhere
3. **Name your agent** (e.g., `security-reviewer.agent.md`)
4. **Fill in the template** – header + instructions

Excercise

Create an agent that will review your projects code base and suggest improvements in a markdown file called review.md.

Quick Tips

- Start simple – add complexity as needed
- Test your agent – does it behave as expected?
- Iterate on instructions – refine based on outputs
- Use descriptive names – `api-designer` not `agent1`
- Don’t overload with tools – less is more
- Don’t write essays – keep instructions scannable

Summary

Concept	Key Takeaway
Custom agents	Configure Copilot for specific tasks/personas
File location	`.github/agents/*.agent.md`
Header	YAML frontmatter with tools, model, handoffs
Body	Markdown instructions for AI behavior
Best practice	Be specific, set boundaries, keep concise

Resources

- VS Code Custom Agents Docs
- GitHub Custom Instructions Docs
- Awesome Copilot Examples

Questions?